

Mitigating Gradient Decay in Soft Decision Trees via Depth-Dependent Gradient Scaling

Xinyu Ni Will Degroot
Dartmouth College

Abstract

Soft decision trees enable gradient-based training of decision trees via probabilistic routing, but gradient magnitudes decay exponentially with depth, limiting model complexity. We analyze this vanishing gradient problem and evaluate three mitigation strategies: localized loss, mixed loss, and depth-dependent gradient scaling. The localized and mixed approaches fail to produce trees beyond the truncation depth, while gradient scaling stabilizes training across all depths.

1 Soft Decision Tree Preliminaries

In an ordinary hard decision tree, each internal node applies a threshold rule and routes an input entirely to the left or to the right child. A soft decision tree replaces this hard routing rule with a probabilistic routing function. At each node m , we assign a routing probability $g_m(\mathbf{x})$ given by a sigmoid applied to a linear transformation of the input:

$$\sigma(z) = 1/(1 + e^{-z})$$

where $z = w_m^T \mathbf{x} + b_m$.

Let $\mathbf{x} = \{x_1, \dots, x_d\} \in \mathbb{R}^d$ denote an input vector. We index nodes in the tree by m , and write m_L and m_R for the immediate left and right children of m . Each internal node m is associated with parameters w_m and b_m . The gating function is

$$g_m(x) = 1/e^{-(w_m^T x + b)} \in [0, 1].$$

The probability of going right given $\mathbf{x}$ at node m is $P(R|\mathbf{x}) = g_m(\mathbf{x})$, while the probability of going left is $P(L|\mathbf{x}) = 1 - g_m(\mathbf{x})$.

This definition corresponds to Figure 1 of Irsoy et al. (2012), where the function $F_m(\mathbf{x})$ is defined recursively. If m is a leaf node $m > \text{root}$, the leaf value $y = z_m$ is returned. Otherwise, if m is not a leaf node,

$$g_m(\mathbf{x}) = 1/(1 + \exp(-(w_m^T \mathbf{x} + b_m))) \quad (1)$$

and the subtree output at m is

$$F_m(\mathbf{x}) = g_m(\mathbf{x})F_{m_L}^L(\mathbf{x}) + (1 - g_m(\mathbf{x}))F_{m_R}^R(\mathbf{x}) \quad (2)$$

Let L denote the set of leaves of a given tree. The

probability of arriving at a particular leaf ℓ given $\mathbf{x}$ is

$$\mu_\ell(\mathbf{x}) = \left(\prod_{m \in \mathcal{P}(\ell)^l} g_m(\mathbf{x}) \right) \left(\prod_{m \in \mathcal{P}(\ell)^r} (1 - g_m(\mathbf{x})) \right) \quad (3)$$

where $\mathcal{P}(\ell)^l$ is the set of all left nodes on the unique path to leaf ℓ , and $\mathcal{P}(\ell)^r$ is the set of all right nodes on this path. In other words, we begin at the root and move toward ℓ , multiplying by $g_m(\mathbf{x})$ whenever the path goes left and by $1 - g_m(\mathbf{x})$ whenever the path goes right. We will use the same notation for internal nodes, writing $\mu_m(X)$ for the path probability of reaching node m from the root. The overall tree output is the expected value over all leaves:

$$F_{\text{root}}(\mathbf{x}) = \sum_{\ell} \mu_\ell(\mathbf{x}) z_\ell \quad (4)$$

A soft decision tree may be used for either classification or regression. In regression, the standard loss is squared error; in classification, we use cross-entropy loss after passing the root output through a sigmoid. Given target r and output y , these losses are

$$E = \begin{cases} \frac{1}{2}(r - y)^2 & \text{Regression} \\ -[r \log y + (1 - r) \log(1 - y)] & \text{Classification} \end{cases} \quad (5)$$

For the remainder of this paper, we focus primarily on the classification setting.

2 Dataset and Preprocessing

All experiments were conducted on the UCI Bike Sharing dataset. This dataset records hourly bike rental counts together with calendar, seasonal, and weather-related features. In particular, the raw feature set includes season,

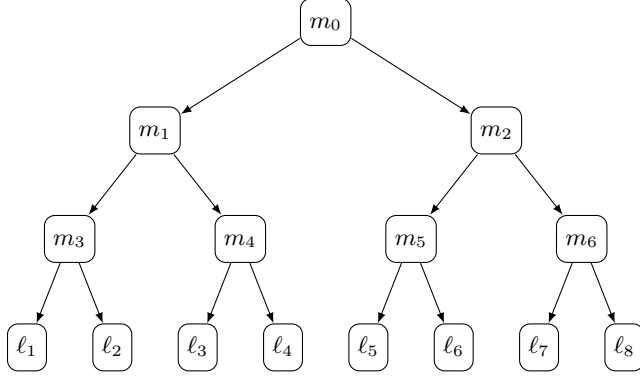


Figure 1: A depth-3 soft decision tree with internal nodes m_0 - m_6 and leaves ℓ_1 - ℓ_8 .

year, month, hour, holiday status, weekday, working day status, weather situation, temperature, “feels like” temperature, humidity, and windspeed. The original response variable is the hourly bike rental count.

Since our focus in this paper is binary classification rather than regression, we converted the count response into a binary label. Specifically, if the hourly count was greater than or equal to the 75th percentile of the observed counts, then we labeled that sample as positive; otherwise, we labeled it as negative. This produces a high-demand versus non-high-demand classification problem.

Before training, we applied a simple preprocessing pipeline. First, we removed the date field `dteday`, since our implementation did not directly encode raw date strings. Next, we one-hot encoded the discrete variables `season`, `mnth`, `hr`, `weekday`, and `weathersit`, yielding a fully numerical feature matrix. We then converted the processed dataset into a NumPy array.

Finally, we split the data randomly into training, validation, and test sets using a 60/20/20 split. The training set was used for stochastic gradient updates, the validation set was used in the iterative tree-building procedure to determine whether candidate splits should be accepted, and the test set was held out for final evaluation. For the main comparison in this paper, we standardized features using only the mean and standard deviation of the training split. This ensured that the ordinary and scaled soft decision tree models were compared on the same representation of the data.

3 Iterative Tree-Building

We follow an inductive tree-building procedure in the spirit of Irsoy et al. (2012). The tree begins as a single leaf, and candidate splits are introduced one node at a time. At each stage, we choose a current leaf node m , attach two new leaf children, and optimize the parameters associated with that split using stochastic gradient descent. If the resulting split decreases validation error, then the split is

accepted and the children may themselves be expanded recursively. Otherwise, the node m is restored as a leaf.

This procedure is greedy rather than globally optimal, but it is simple to implement and naturally couples parameter fitting with tree-structure selection. It is also the mechanism that allows us to study whether different gradient modifications change not only the values learned within the tree, but also the eventual depth and shape of the tree that is grown.

4 Ordinary Gradient Descent Model

To construct a soft decision tree classifier, we must determine both the structure of the tree and the values of the gate and leaf parameters. In this section, we describe the ordinary gradient descent approach of Irsoy et al. (2012), which addresses both tasks simultaneously through an inductive tree-building scheme.

Fix a soft decision tree $\mathcal{T}$ and a node $m \in \mathcal{T}$ connected to two leaves with values z_m^L and z_m^R . Suppose that we only allow the quantities g_m , z_m^L , and z_m^R to vary, while holding the remainder of the tree fixed. Then it is natural to apply gradient descent to the parameter set

$$\theta = \{w_m, b_m, z_m^L, z_m^R\}.$$

We carry this out using stochastic gradient descent. Let $\mathcal{X}$ denote the training set, and let $(x, r) \in \mathcal{X}$. In the classification setting, the error function is

$$E = -[r \log(\sigma(F_{root}(x))) + (1-r) \log(1-\sigma(F_{root}(x)))]$$

where σ is a sigmoid function centered at the origin. For a given parameter set θ , we seek the quantities

$$\frac{\partial E}{\partial w_m}, \quad \frac{\partial E}{\partial b_m}, \quad \frac{\partial E}{\partial z_m^L}, \quad \frac{\partial E}{\partial z_m^R}.$$

Starting with w_m , fix $j \in \{0, \dots, d\}$ and observe:

$$\frac{\partial E}{\partial w_{mj}} = \frac{\partial E}{\partial F_m(X)} \cdot \frac{\partial F_m(X)}{\partial g_m(X)} \cdot \frac{\partial g_m(X)}{\partial w_{mj}}.$$

We first claim that

$$\frac{\partial E}{\partial F_m(X)} = (\sigma(F_{root}(X)) - r) \mu_m(x).$$

Indeed,

$$\frac{\partial E}{\partial F_{root}(X)} = \sigma(F_{root}(X)) - r$$

and

$$\frac{\partial F_{root}(X)}{\partial F_m(X)} = \mu_m(x),$$

so the claim follows by the chain rule. We write this error signal as

$$\delta_m := \frac{\partial E}{\partial F_m(x)}$$

for convenience. By definition,

$$\frac{\partial F_m(x)}{\partial g_m(x)} = F_m^L(x) - F_m^R(x),$$

and one can also check that

$$\frac{\partial g_m(x)}{\partial w_{mj}} = g_m(x)(1 - g_m(x))x_j.$$

Combining these terms yields

$$\frac{\partial E}{\partial w_{mj}} = \delta_m(F_m^L(\mathbf{x}) - F_m^R(\mathbf{x}))g_m(x)(1 - g_m(x))x_j \quad (6)$$

The same derivation gives the corresponding gradient for b_m .

Turning now to z_L and z_R , note from (4) that

$$\frac{\partial F_{root}}{\partial z_\ell} = \mu_\ell(x) \quad (7)$$

Hence, by the chain rule,

$$\frac{\partial E}{\partial z_\ell} = \frac{\partial E}{\partial F_{root}} \frac{\partial F_{root}}{\partial z_\ell} = \delta_{root} \mu_\ell(x) \quad (8)$$

In all of our experiments, we trained this ordinary gradient descent model using stochastic gradient descent, at 10 epochs per node, with a learning rate of .01.

5 The Vanishing Gradient Problem

In ordinary gradient descent, for a fixed node m , the error signal δ_m contains the term $\mu_x(m)$. If we assume that $g_{m'}(x) \approx .5$, averaging over all nodes m' in the tree, then we should expect that

$$\mu_x(m) \approx \left(\frac{1}{2}\right)^{\text{depth}(m)}.$$

A consequence is that all gradients with respect to the parameters θ_m decrease exponentially, tending toward zero as the depth of the tree increases. This is problematic because it means that deeper nodes have a negligible effect on the output, functionally limiting the complexity of the model. In addition to this compounding effect from gate probabilities, one should also note that as training progresses and the model becomes more accurate, the term $\sigma(F_{root}(x)) - r$ also shrinks. This further contributes to vanishing gradients.

The purpose of this project is to investigate the following stated approaches aimed at keeping gradients consistently and meaningfully large throughout the entire tree. In all experiments, we measured the size of the gradient by computing its norm.

6 Localized Gradient Descent Model

The localized gradient descent model differs from the ordinary model only in that the error function used to optimize θ_m depends on the node m . Specifically, to perform gradient descent on

$$\theta_m = \{w_m, b_m, z_m^L, z_m^R\},$$

we use the error function

$$E_m = [r \log(\sigma(F_{root'}(x))) + (1-r) \log(1 - \sigma(F_{root'}(x)))]$$

where $root' = m.$ $\overbrace{\text{parent.parent} \cdots \text{parent}}^{n \text{ times}}$ for some parameter n (truncating if we hit the root). We refer to this parameter n as the *mod_depth*.

Repeating the derivation from the previous section, all gradient formulas remain unchanged except that the error signal δ_m is replaced by

$$\delta_m^n = (\sigma(F_{root'}(X)) - r) \mu_m^n(x)$$

where μ_m^n denotes the probability weight of the unique path from $root'$ to m . We trained the localized gradient descent model using stochastic gradient descent for 10 epochs at each node with learning rate .01.

7 Mixed Gradient Descent Model

To interpolate between the ordinary and localized approaches, we define a mixed error function

$$E_m^\lambda = \lambda \cdot E + (1 - \lambda) \cdot E_m.$$

The mixed gradient follows from linearity:

$$\frac{\partial E_m^\lambda}{\partial \theta_m} = \lambda \frac{\partial E}{\partial \theta_m} + (1 - \lambda) \frac{\partial E_m}{\partial \theta_m}.$$

Again in the mixed case, we trained for 10 epochs using a learning rate of .01.

8 The Scaling Model

In this final model, we performed ordinary gradient descent, except that we scaled the gradient according to

$$\frac{\partial E}{\partial \theta_m} := 2^{\text{depth}(m)} \frac{\partial E}{\partial \theta_m}$$

before subtracting $\eta \cdot \frac{\partial E}{\partial \theta_m}$ from θ_m .

9 Results and Discussion

The motivation behind our localized gradient model is that

$$\mu_x^{\text{mod_depth}}(m) \leq \left(\frac{1}{2}\right)^{\text{mod_depth}}$$

for all nodes m in the tree. In other words, the exponential gradient decay is bounded by the *mod_depth* constant.

Moreover, we initially hypothesized that optimizing the local error function would still decrease the global loss. This seemed plausible because, in many situations, different subtrees of a decision tree behave somewhat independently. For example, in a decision tree that classifies whether an image depicts a pig, one might expect that the relationship between *pink-ness* and *pig-ness* could be optimized somewhat independently from the rest of the tree.

However, this intuition was not supported in our localized gradient experiments. In every case we tested, performing localized gradient descent at a node m with $\text{depth}(m) \geq \text{mod_depth} + 1$ failed to decrease the global error. Under our iterative tree-building scheme, this implies that the localized approach failed to produce a tree of $\text{depth} > \text{mod_depth} + 1$. But if the localized approach terminates at a $\text{depth} \leq \text{mod_depth} + 1$, then by definition this is exactly the same tree that would have been produced by the ordinary approach with maximum depth $\text{mod_depth} + 1$. Hence, the localized experiment did not yield any new empirical advantage.

In response to this early truncation behavior, we constructed a mixed gradient descent model that blends the local and global objectives using a parameter λ . When $\lambda = 1$, the mixed model coincides with the ordinary model; when $\lambda = 0$, it reduces to the fully localized model. Our goal was to construct a hybrid method that would reduce vanishing gradients without entirely discarding the global error function E in favor of the local error function E_m . However, this was still insufficient: even at high values such as $\lambda = .8$ and $\lambda = .9$, the model continued to fail to build a tree of $\text{depth} > \text{mod_depth} + 1$. Our conclusion is that local error on a soft decision tree is not a viable substitute for a single global objective when working with real-world, multi-dimensional data.

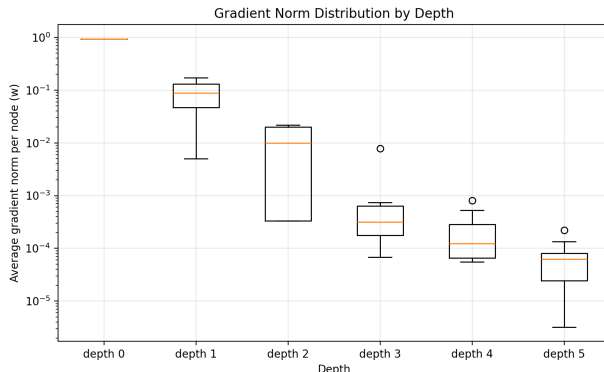


Figure 2: Gradient norm distribution by depth for the ordinary soft decision tree on the Bike Sharing Dataset. The downward trend is consistent with vanishing gradients in deeper nodes.

The final idea we tested was to scale the gradient di-

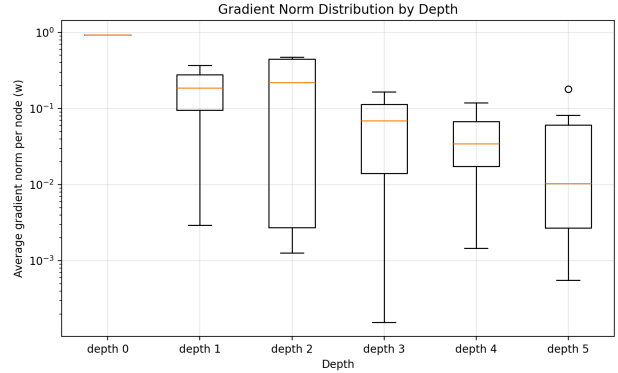


Figure 3: Compared to the ordinary model, the gradient magnitudes remain more stable across depth post-scaling.

rectly in response to the theoretical decay rate $(\frac{1}{2})^{\text{depth}(m)}$. This scaling model kept the median gradient norm roughly between norms of 10^{-2} and 10^{-1} , and it concretely improved the classification metrics we measured. In our final run, logistic regression achieved a test accuracy of 87.4%, the ordinary soft decision tree achieved 90.4%, and the scaled soft decision tree achieved 93.4%. The same ordering held across precision, recall, F1 score, and log-loss. This is consistent with the motivating hypothesis of the paper: although the ordinary soft decision tree already outperforms a linear baseline, compensating for depth-dependent attenuation allows the tree to train more effectively and yields a stronger classifier.

Table 1: Classification metrics on the UCI Bike Sharing dataset.

Model	Acc.	Prec.	Rec.	F1	Log-Loss
Logistic Regression	0.8737	0.7601	0.7151	0.7370	0.2713
Ordinary SDT	0.9039	0.8044	0.8081	0.8063	0.2255
Scaled SDT	0.9336	0.8849	0.8407	0.8623	0.1643

The success of the scaling procedure suggests several directions for future work. First, in practice the gate values $g_m(x)$ vary substantially across nodes and samples, and there is no guarantee that they meaningfully average to $\frac{1}{2}$. Consequently, it is not clear that $\beta = 2$ is the optimal choice when scaling by $\beta^{\text{depth}(m)}$ at each step of gradient descent. Determining a better value of β or a better depth-based scaling function would be of interest. Second, many localized training schemes remain unexplored; for example, one might ask whether some combination of scaling and a local or hybrid objective could perform better than either modification alone. Finally, our conclusions are based on only two datasets and a narrow choice of hyperparameters. A broader empirical study across more datasets and training settings would provide additional context for the conclusions presented here.

References

- [1] Irsoy, O., Yildiz, O.T. & Alpaydin, E. (2012) Soft decision trees. In *Proceedings of the 21st International Conference on Pattern Recognition (ICPR)*, pp. 1819–1822.
- [2] Frosst, N. & Hinton, G. (2017) Distilling a neural network into a soft decision tree. *arXiv preprint arXiv:1711.09784*.
- [3] Bishop, C. M. (2006) *Pattern Recognition and Machine Learning*. Springer.
- [4] Bengio, Y., Simard, P. & Frasconi, P. (1994) Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks* **5**(2):157–166.
- [5] Fivel, O., Klein, M. & Maimon, O. (2022) Decision trees with soft numbers. *arXiv preprint arXiv:2201.03414*.
- [6] Kotschieder, P., Fiterau, M., Criminisi, A. & Buló, S. R. (2015) Deep neural decision forests. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1467–1475.
- [7] Fanaee-T, H. (2013) Bike Sharing Dataset. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5W894>.
- [8] Jordan, M. I. & Jacobs, R. A. (1994) Hierarchical mixtures of experts and the EM algorithm. *Neural Computation* **6**(2):181–214.